

Input Validation and CSRF Protection

Section: 8 — Request/Response Handling, Forms, and Validation

Subtopic: 03 — Validation

Estimated Duration: 2–3 hours

Theory

Practical

Topics covered: Why validation matters, manual validation pitfalls, express-validator setup, validation chains, sanitisation, custom validators, validation middleware patterns, error formatting, Joi as an alternative, CSRF protection with csrf

Learning Objectives — This Subtopic

- Explain why server-side validation is essential regardless of client-side checks
- Install and configure `express-validator` to validate incoming request data
- Build validation chains that check, sanitise, and produce structured error responses
- Create reusable validation middleware for common patterns (registration, login, resource creation)
- Use Joi as an alternative schema-based validation library and compare the two approaches
- Implement CSRF protection using the `csrf-csrf` package to guard against cross-site request forgery

1. Why Server-Side Validation Matters

Every value arriving in `req.body`, `req.query`, or `req.params` is untrusted input. Client-side validation (HTML5 attributes, JavaScript checks in the browser) improves user experience, but it provides **zero security**. Any request can be crafted directly with tools like `curl`, Postman, or a malicious script — bypassing the browser entirely.

Server-side validation guarantees three things:

- **Data integrity** — Only well-formed, expected data reaches your business logic and database.
- **Security** — Malicious payloads (SQL injection fragments, XSS scripts, oversized strings) are rejected before they cause harm.
- **Consistency** — Every API consumer (web app, mobile app, third-party integration) is subject to the same rules.

Analogy: Think of validation as airport security. A passenger (the client) might claim they have a valid boarding pass, but the gate agent (the server) still scans it. No matter how trustworthy the passenger appears, the check always happens at the gate.

1.1 The Problem with Manual Validation

Before introducing a library, consider what manual validation looks like. This makes the value of a dedicated tool immediately obvious.

Start with a simple Express app. Create a new project directory and initialise it:

```
mkdir validation-demo && cd validation-demo
```

```
npm init -y
```

```
npm install express
```

```
validation-demo/  
├─ node_modules/  
├─ package.json  
└─ server.js
```

Stage 1 Manual validation — naive approach `server.js`

```
const express = require('express');  
const app = express();  
  
app.use(express.json());  
  
app.post('/register', (req, res) => {  
  const { username, email, password } = req.body;  
  
  // Manual checks  
  const errors = [];  
  
  if (!username || typeof username !== 'string') {  
    errors.push('Username is required and must be a string.');  } else if (username.trim().length < 3) {  
    errors.push('Username must be at least 3 characters.');  } else if (username.trim().length > 30) {  
    errors.push('Username must be at most 30 characters.');  }  
  
  if (!email || typeof email !== 'string') {  
    errors.push('Email is required.');  } else if (!/^([^\s@]+@[^\s@]+\.[^\s@]+)$/.test(email)) {  
    errors.push('Email must be a valid email address.');  }  
  
  if (!password || typeof password !== 'string') {  
    errors.push('Password is required.');  } else if (password.length < 8) {  
    errors.push('Password must be at least 8 characters.');  }  
  
  if (errors.length > 0) {  
    return res.status(400).json({ errors });  
  }  
  
  // If we reach here, data is valid  
  res.status(201).json({
```

```
    message: 'User registered',
    user: { username: username.trim(), email: email.toLowerCase() }
  });
});

app.listen(3000, () => {
  console.log('Server running on http://localhost:3000');
});
```

Test it with a missing field:

```
node server.js
```

In another terminal:

```
curl -X POST http://localhost:3000/register -H "Content-Type: application/json" -d '{"username": "ab", "email": "not-an-email", "password": "short"}'
```

Output:

```
{
  "errors": [
    "Username must be at least 3 characters.",
    "Email must be a valid email address.",
    "Password must be at least 8 characters."
  ]
}
```

This works, but look at the cost: roughly 20 lines of `if/else` logic for three fields. A real registration form might have ten or more fields, each with multiple rules. The code becomes brittle, repetitive, and difficult to maintain. This is exactly the problem validation libraries solve.

Warning: Never rely on client-side validation alone. Any HTTP client can bypass it. Always validate on the server, even if the same checks exist in the browser.

2. express-validator — Setup and Core Concepts

`express-validator` is the most widely used validation library in the Express ecosystem. It wraps the well-established `validator.js` library and integrates directly into the middleware pipeline.

Install it:

```
npm install express-validator
```

2.1 How It Works — The Three-Step Pattern

Every validation with `express-validator` follows three steps:

1. **Define validation chains** — Declare rules for each field using chainable methods.
2. **Run the chains as middleware** — Pass them as an array in the route handler.
3. **Check for errors** — Call `validationResult(req)` to collect any failures.

Stage 2 First express-validator usage `server.js`

```
const express = require('express');
const { body, validationResult } = require('express-validator');

const app = express();
app.use(express.json());

app.post(
  '/register',
  [
    body('username')
      .trim()
      .notEmpty().withMessage('Username is required.')
      .isLength({ min: 3, max: 30 }).withMessage('Username must be 3-30 characters.'),

    body('email')
      .normalizeEmail()
      .isEmail().withMessage('Must be a valid email address.'),

    body('password')
      .isLength({ min: 8 }).withMessage('Password must be at least 8 characters.')
  ],
  (req, res) => {
    const errors = validationResult(req);
```

```
if (!errors.isEmpty()) {  
    return res.status(400).json({ errors: errors.array() });  
}  
  
res.status(201).json({  
    message: 'User registered',  
    user: { username: req.body.username, email: req.body.email }  
});  
}  
);  
  
app.listen(3000, () => {  
    console.log('Server running on http://localhost:3000');  
});
```

Test with the same invalid data:

```
curl -X POST http://localhost:3000/register -H "Content-Type: application/json" -d '{"username":  
"ab", "email": "not-an-email", "password": "short"}'
```

Output:

```
{  
  "errors": [  
    {  
      "type": "field",  
      "value": "ab",  
      "msg": "Username must be 3-30 characters.",  
      "path": "username",  
      "location": "body"  
    },  
    {  
      "type": "field",  
      "value": "not-an-email",  
      "msg": "Must be a valid email address.",  
      "path": "email",  
      "location": "body"  
    },  
    {  
      "type": "field",  
      "value": "short",  
      "msg": "Password must be at least 8 characters.",  
      "path": "password",  
      "location": "body"  
    }  
  ]  
}
```

```
}
]
}
```

What is happening?

- `body('username')` targets the `username` field in `req.body`. The equivalent functions `query()` and `param()` target `req.query` and `req.params` respectively.
- `.trim()` is a **sanitiser** — it modifies the value (strips whitespace) before subsequent validators run.
- `.notEmpty()` and `.isLength()` are **validators** — they check conditions and add errors if they fail.
- `.withMessage()` attaches a custom error message to the validator immediately before it.
- `.normalizeEmail()` lowercases the domain and applies common normalisations (e.g., removing dots from Gmail addresses).
- `validationResult(req)` collects all errors from all chains that ran against this request.
- Each error object includes the field `path`, the invalid `value`, the `msg`, and the `location` (body, query, params).

Compare this with the manual version: the same logic is expressed in far fewer lines, the error format is standardised, and adding new fields or rules requires minimal effort.

Key Insight: Sanitisers like `.trim()`, `.normalizeEmail()`, and `.escape()` modify `req.body` in place. This means your route handler receives clean data — no need to manually trim or lowercase values.

3. Validation Chains in Depth

3.1 Common Validators

`express-validator` exposes all validators from `validator.js`. Here are the ones used most frequently:

Validator	Purpose	Example
-----------	---------	---------

<code>.notEmpty()</code>	Rejects empty strings	<code>body('name').notEmpty()</code>
<code>.isEmail()</code>	Validates email format	<code>body('email').isEmail()</code>
<code>.isLength({ min, max })</code>	Checks string length	<code>body('bio').isLength({ max: 500 })</code>
<code>.isInt({ min, max })</code>	Integer within range	<code>body('age').isInt({ min: 18 })</code>
<code>.isFloat({ min, max })</code>	Floating-point within range	<code>body('price').isFloat({ min: 0 })</code>
<code>.isBoolean()</code>	Must be <code>true</code> or <code>false</code>	<code>body('agree').isBoolean()</code>
<code>.isIn([...])</code>	Must be one of the given values	<code>body('role').isIn(['admin', 'user'])</code>
<code>.matches(regex)</code>	Tests against a regular expression	<code>body('slug').matches(/^[a-z0-9-]+\$ /)</code>
<code>.isURL()</code>	Validates URL format	<code>body('website').isURL()</code>
<code>.isISO8601()</code>	Validates ISO date string	<code>body('dob').isISO8601()</code>

3.2 Common Sanitisers

Sanitiser	Effect
<code>.trim()</code>	Removes leading/trailing whitespace
<code>.escape()</code>	HTML-encodes <code><</code> , <code>></code> , <code>&</code> , <code>'</code> , <code>"</code> , <code>/</code>
<code>.normalizeEmail()</code>	Lowercases domain, standardises Gmail dots
<code>.toInt()</code>	Converts to integer
<code>.toFloat()</code>	Converts to float
<code>.toBoolean()</code>	Converts to boolean

```
.blacklist(chars)
```

Removes specified characters

3.3 Chaining Validators and Sanitisers Together

Chains execute in order. Place sanitisers *before* validators that depend on the sanitised value.

Stage 3 Product creation route with mixed validators and sanitisers `server.js`

```
const express = require('express');
const { body, validationResult } = require('express-validator');

const app = express();
app.use(express.json());

app.post(
  '/products',
  [
    body('name')
      .trim()
      .notEmpty().withMessage('Product name is required.')
      .isLength({ min: 2, max: 100 }).withMessage('Name must be 2-100 characters.')
      .escape(),

    body('price')
      .notEmpty().withMessage('Price is required.')
      .isFloat({ min: 0.01 }).withMessage('Price must be a positive number.')
      .toFloat(),

    body('category')
      .trim()
      .notEmpty().withMessage('Category is required.')
      .isIn(['electronics', 'clothing', 'food', 'books'])
      .withMessage('Category must be one of: electronics, clothing, food, books.'),

    body('inStock')
      .optional()
      .isBoolean().withMessage('inStock must be true or false.')
      .toBoolean()
  ],
  (req, res) => {
    const errors = validationResult(req);

    if (!errors.isEmpty()) {
      return res.status(400).json({ errors: errors.array() });
    }
  }
);
```

```
}

// req.body.price is now a number (toFloat applied)
// req.body.name has been trimmed and HTML-escaped
res.status(201).json({ message: 'Product created', product: req.body });
}
);

app.listen(3000, () => {
  console.log('Server running on http://localhost:3000');
});
```

```
curl -X POST http://localhost:3000/products -H "Content-Type: application/json" -d '{"name": "Laptop ", "price": "999.99", "category": "electronics"}'
```

Output:

```
{
  "message": "Product created",
  "product": {
    "name": "Laptop",
    "price": 999.99,
    "category": "electronics"
  }
}
```

What is happening?

- `.trim()` stripped the whitespace from `" Laptop "`, resulting in `"Laptop"`.
- `.escape()` ran after trimming — since the name contained no HTML characters, it had no visible effect. But if a user submitted `<script>alert('xss')</script>`, it would be encoded safely.
- `.toFloat()` converted the string `"999.99"` to the number `999.99`.
- `.optional()` on `inStock` means the field can be omitted entirely without triggering an error. When present, it must pass `.isBoolean()`.

Now test with invalid data:

```
curl -X POST http://localhost:3000/products -H "Content-Type: application/json" -d '{"name": "", "price": "-5", "category": "furniture"}'
```

Output:

```
{
  "errors": [
    {
      "type": "field",
      "value": "",
      "msg": "Product name is required.",
      "path": "name",
      "location": "body"
    },
    {
      "type": "field",
      "value": "-5",
      "msg": "Price must be a positive number.",
      "path": "price",
      "location": "body"
    },
    {
      "type": "field",
      "value": "furniture",
      "msg": "Category must be one of: electronics, clothing, food, books.",
      "path": "category",
      "location": "body"
    }
  ]
}
```

4. Custom Validators

Built-in validators cover most cases, but some rules are specific to your application. The `.custom()` method accepts a function that either returns `true`, throws an error, or returns a rejected promise.

Stage 4 Custom validator — password confirmation `server.js`

```
const express = require('express');
const { body, validationResult } = require('express-validator');

const app = express();
app.use(express.json());

app.post(
  '/register',
```

```
[
  body('username')
    .trim()
    .notEmpty().withMessage('Username is required.')
    .isLength({ min: 3, max: 30 }).withMessage('Username must be 3-30 characters.'),

  body('email')
    .normalizeEmail()
    .isEmail().withMessage('Must be a valid email address.'),

  body('password')
    .isLength({ min: 8 }).withMessage('Password must be at least 8 characters.')
    .matches(/[A-Z]/).withMessage('Password must contain at least one uppercase letter.')
    .matches(/[0-9]/).withMessage('Password must contain at least one number.'),

  body('confirmPassword')
    .custom((value, { req }) => {
      if (value !== req.body.password) {
        throw new Error('Password confirmation does not match.');      }
      return true;
    })
],
(req, res) => {
  const errors = validationResult(req);

  if (!errors.isEmpty()) {
    return res.status(400).json({ errors: errors.array() });
  }

  res.status(201).json({
    message: 'User registered',
    user: { username: req.body.username, email: req.body.email }
  });
}
);

app.listen(3000, () => {
  console.log('Server running on http://localhost:3000');
});
```

```
curl -X POST http://localhost:3000/register -H "Content-Type: application/json" -d '{"username":
"alice", "email": "alice@example.com", "password": "Secret123", "confirmPassword": "Secret456"}'
```

Output:

```
{
  "errors": [
    {
      "type": "field",
      "value": "Secret456",
      "msg": "Password confirmation does not match.",
      "path": "confirmPassword",
      "location": "body"
    }
  ]
}
```

What is happening?

- The `.custom()` callback receives the field's value as its first argument and a context object as its second. The context includes `req`, which gives access to other fields.
- Throwing an `Error` inside the callback causes the validator to fail with that error's message.
- Returning `true` signals the validation passed.

Note: Custom validators can also be asynchronous. Return a `Promise` — if it rejects, the validation fails. This is useful for checking uniqueness against a database (e.g., "is this email already taken?").

5. Reusable Validation Middleware

As an application grows, repeating validation arrays and the `validationResult` check in every route becomes tedious. Extract both into reusable modules.

Stage 5 Organised validation with separate files

```
validation-demo/
├─ node_modules/
├─ middleware/
└─ validate.js
```

```
├─ validators/  
│   └─ userValidators.js  
├─ package.json  
└─ server.js
```

First, create a generic validation-runner middleware. This middleware checks for errors and returns a standardised 400 response if any exist.

middleware/validate.js

```
const { validationResult } = require('express-validator');  
  
const validate = (req, res, next) => {  
  const errors = validationResult(req);  
  
  if (!errors.isEmpty()) {  
    return res.status(400).json({  
      status: 'error',  
      errors: errors.array().map(err => ({  
        field: err.path,  
        message: err.msg  
      })))  
    });  
  }  
  
  next();  
};  
  
module.exports = validate;
```

Next, define the validation rules for user-related routes in a dedicated file:

validators/userValidators.js

```
const { body } = require('express-validator');  
  
const registerRules = [  
  body('username')  
    .trim()  
    .notEmpty().withMessage('Username is required.')  
    .isLength({ min: 3, max: 30 }).withMessage('Username must be 3-30 characters.')  
    .matches(/^[a-zA-Z0-9_]+$/).withMessage('Username may only contain letters, numbers, and underscores.'),  
];
```

```
body('email')
  .normalizeEmail()
  .isEmail().withMessage('Must be a valid email address. '),

body('password')
  .isLength({ min: 8 }).withMessage('Password must be at least 8 characters.')
  .matches(/[A-Z]/).withMessage('Password must contain at least one uppercase letter.')
  .matches(/[0-9]/).withMessage('Password must contain at least one number. '),

body('confirmPassword')
  .custom((value, { req }) => {
    if (value !== req.body.password) {
      throw new Error('Password confirmation does not match. ');
    }
    return true;
  })
];

const loginRules = [
  body('email')
    .normalizeEmail()
    .isEmail().withMessage('Must be a valid email address. '),

  body('password')
    .notEmpty().withMessage('Password is required.')
];

module.exports = { registerRules, loginRules };
```

Finally, wire it together in the server:

server.js

```
const express = require('express');
const validate = require('./middleware/validate');
const { registerRules, loginRules } = require('./validators/userValidators');

const app = express();
app.use(express.json());

app.post('/register', registerRules, validate, (req, res) => {
  res.status(201).json({
    message: 'User registered',
    user: { username: req.body.username, email: req.body.email }
  });
});
```

```
});  
});  
  
app.post('/login', loginRules, validate, (req, res) => {  
  res.json({ message: 'Login successful (placeholder)' });  
});  
  
app.listen(3000, () => {  
  console.log('Server running on http://localhost:3000');  
});
```

```
curl -X POST http://localhost:3000/register -H "Content-Type: application/json" -d '{"username":  
"a!", "email": "bad", "password": "weak", "confirmPassword": "different"}'
```

Output:

```
{  
  "status": "error",  
  "errors": [  
    { "field": "username", "message": "Username must be 3-30 characters." },  
    { "field": "username", "message": "Username may only contain letters, numbers, and underscores." },  
    { "field": "email", "message": "Must be a valid email address." },  
    { "field": "password", "message": "Password must be at least 8 characters." },  
    { "field": "password", "message": "Password must contain at least one uppercase letter." },  
    { "field": "password", "message": "Password must contain at least one number." },  
    { "field": "confirmPassword", "message": "Password confirmation does not match." }  
  ]  
}
```

What is happening?

- The route definition reads cleanly: `registerRules` (define checks), then `validate` (run and halt on errors), then the handler (business logic).
- The `validate` middleware is completely generic — it works for any route, any set of rules.
- The error response is normalised: every error has a `field` and a `message`. Frontend consumers can map errors to form fields reliably.
- Adding a new route (e.g., `/products`) requires only a new rules file — the middleware and pattern remain identical.

Key Insight: This pattern — separating rules, a generic validation runner, and route handlers — scales well. In a production Express application, you might have `validators/userValidators.js`, `validators/productValidators.js`, `validators/orderValidators.js`, all sharing the same `validate` middleware.

6. Validating Query Parameters and Route Parameters

So far, every example has validated `req.body`. The same approach works for query strings and route parameters.

server.js (additional routes)

```
const { param, query } = require('express-validator');
const validate = require('./middleware/validate');

// Validate a route parameter
app.get(
  '/users/:id',
  [
    param('id')
      .isInt({ min: 1 }).withMessage('User ID must be a positive integer.')
      .toInt()
  ],
  validate,
  (req, res) => {
    // req.params.id is now a number
    res.json({ message: `Fetching user ${req.params.id}` });
  }
);

// Validate query parameters
app.get(
  '/search',
  [
    query('q')
      .trim()
      .notEmpty().withMessage('Search query is required.')
      .isLength({ min: 2 }).withMessage('Search query must be at least 2 characters.'),
```

```
query('page')
  .optional()
  .isInt({ min: 1 }).withMessage('Page must be a positive integer.')
  .toInt(),

query('limit')
  .optional()
  .isInt({ min: 1, max: 100 }).withMessage('Limit must be between 1 and 100.')
  .toInt()
],
validate,
(req, res) => {
  const { q, page = 1, limit = 20 } = req.query;
  res.json({ query: q, page, limit });
}
);
```

```
curl "http://localhost:3000/users/abc"
```

Output:

```
{
  "status": "error",
  "errors": [
    { "field": "id", "message": "User ID must be a positive integer." }
  ]
}
```

```
curl "http://localhost:3000/search?q=no&page=0"
```

Output:

```
{
  "status": "error",
  "errors": [
    { "field": "page", "message": "Page must be a positive integer." }
  ]
}
```

```
curl "http://localhost:3000/search?q=nodejs&page=2&limit=10"
```

Output:

```
{
  "query": "nodejs",
  "page": 2,
  "limit": 10
}
```

The functions `body()`, `param()`, and `query()` are interchangeable in their API — only the source location differs.

7. Joi — Schema-Based Validation Alternative

Joi takes a different approach. Instead of attaching validation chains to individual fields within a middleware array, Joi defines a **schema object** that describes the entire shape of expected data. The schema is then used to validate the request body (or any object) in one call.

```
npm install joi
```

Stage 6 Joi — basic schema `validators/joiExample.js`

```
const Joi = require('joi');

const registerSchema = Joi.object({
  username: Joi.string()
    .trim()
    .min(3)
    .max(30)
    .pattern(/^[a-zA-Z0-9_]+$/)
    .required()
    .messages({
      'string.empty': 'Username is required.',
      'string.min': 'Username must be at least 3 characters.',
      'string.max': 'Username must be at most 30 characters.',
      'string.pattern.base': 'Username may only contain letters, numbers, and underscores.'
    })
  email: Joi.string()
    .email()
```

```
.required()
.messages({
  'string.email': 'Must be a valid email address.',
  'any.required': 'Email is required.'
}),

password: Joi.string()
  .min(8)
  .required()
  .messages({
    'string.min': 'Password must be at least 8 characters.'
  }),

confirmPassword: Joi.any()
  .valid(Joi.ref('password'))
  .required()
  .messages({
    'any.only': 'Password confirmation does not match.'
  })
});

module.exports = { registerSchema };
```

middleware/joiValidate.js

```
const joiValidate = (schema) => {
  return (req, res, next) => {
    const { error, value } = schema.validate(req.body, { abortEarly: false });

    if (error) {
      const errors = error.details.map(detail => ({
        field: detail.path.join('.'),
        message: detail.message
      }));

      return res.status(400).json({ status: 'error', errors });
    }

    // Replace req.body with the validated (and potentially transformed) value
    req.body = value;
    next();
  };
};

module.exports = joiValidate;
```

server.js (Joi route example)

```
const express = require('express');
const joiValidate = require('./middleware/joiValidate');
const { registerSchema } = require('./validators/joiExample');

const app = express();
app.use(express.json());

app.post('/register-joi', joiValidate(registerSchema), (req, res) => {
  res.status(201).json({
    message: 'User registered (Joi)',
    user: { username: req.body.username, email: req.body.email }
  });
});

app.listen(3000, () => {
  console.log('Server running on http://localhost:3000');
});
```

```
curl -X POST http://localhost:3000/register-joi -H "Content-Type: application/json" -d '{"username":
"a!", "email": "bad", "password": "weak", "confirmPassword": "different"}'
```

Output:

```
{
  "status": "error",
  "errors": [
    { "field": "username", "message": "Username must be at least 3 characters." },
    { "field": "username", "message": "Username may only contain letters, numbers, and underscores." },
    { "field": "email", "message": "Must be a valid email address." },
    { "field": "password", "message": "Password must be at least 8 characters." },
    { "field": "confirmPassword", "message": "Password confirmation does not match." }
  ]
}
```

7.1 express-validator vs Joi — When to Use Which

Criterion	express-validator	Joi
Integration	Native Express middleware — chains sit directly in route definitions	Framework-agnostic — works anywhere, needs a middleware wrapper for Express

Validation source	Can validate <code>body</code> , <code>query</code> , <code>params</code> , <code>headers</code> , <code>cookies</code> independently	Validates a single object (typically <code>req.body</code>); separate schemas needed per source
Style	Imperative chains per field	Declarative schema for the entire object
Custom messages	<code>.withMessage()</code> per validator	<code>.messages()</code> object with error type keys
Sanitisation	Built-in sanitisers in the same chain	Transformations via <code>.trim()</code> , <code>.lowercase()</code> , <code>.def</code>
Ecosystem	Very common in Express tutorials and production apps	Popular in Hapi ecosystem, also widely used with Express

Key Insight: Both libraries are production-grade. Choose `express-validator` if you want tight Express integration and per-source validation. Choose Joi if you prefer defining data shapes as standalone schemas that can be reused outside of Express (e.g., in tests, CLI tools, or other frameworks).

8. CSRF Protection

Cross-Site Request Forgery (CSRF) is an attack where a malicious website tricks a user's browser into making an unwanted request to your application. If the user is logged in, the browser automatically attaches their session cookie, and the server cannot distinguish the forged request from a legitimate one.

Analogy: Imagine you leave your signed chequebook on the counter at a coffee shop. Someone slips in a pre-filled cheque and hands it to the cashier. The cashier sees your valid signature (the session cookie) and processes it. CSRF protection is the equivalent of requiring a unique code on every cheque that only you know at the time of writing.

8.1 How CSRF Tokens Work

The defence against CSRF relies on a **token** that the server generates and embeds in the page. When the form is submitted, the token is sent back with the request. A malicious third-party site cannot obtain this token (due to the browser's same-origin policy), so its forged requests are rejected.

The flow is:

1. Server generates a unique CSRF token per session (or per request).
2. Server sends the token to the client (in a hidden form field, a meta tag, or a cookie).
3. Client includes the token in subsequent state-changing requests (POST, PUT, DELETE).
4. Server verifies the token matches. If not, the request is rejected with `403 Forbidden`.

Important: CSRF protection is primarily relevant for **server-rendered applications** that use session cookies. Pure API servers that use token-based authentication (e.g., JWT in the `Authorization` header) are generally not vulnerable to CSRF because the browser does not automatically attach custom headers.

8.2 Implementing CSRF Protection with `csrf-csrf`

The original `csrf` package has been deprecated. The community replacement is `csrf-csrf`, which uses the Double Submit Cookie pattern.

```
npm install csrf-csrf cookie-parser
```

Stage 7 CSRF protection — basic setup `server.js`

```
const express = require('express');
const cookieParser = require('cookie-parser');
const { doubleCsrf } = require('csrf-csrf');

const app = express();

app.use(express.urlencoded({ extended: true }));
app.use(cookieParser('my-secret-key'));

// Configure CSRF protection
const {
  generateToken,
```

```
doubleCsrfProtection
} = doubleCsrf({
  getSecret: () => 'my-csrf-secret-at-least-32-chars-long!',
  cookieName: '__csrf',
  cookieOptions: {
    httpOnly: true,
    sameSite: 'strict',
    secure: false // Set to true in production (HTTPS)
  },
  getTokenFromRequest: (req) => req.body._csrf || req.headers['x-csrf-token']
});

// — Route: Serve the form (GET) —
app.get('/transfer', (req, res) => {
  const csrfToken = generateToken(req, res);

  res.send(`
    <h2>Bank Transfer</h2>
    <form method="POST" action="/transfer">
      <input type="hidden" name="_csrf" value="${csrfToken}" />
      <label>Recipient: <input name="recipient" /></label><br/>
      <label>Amount: <input name="amount" type="number" /></label><br/>
      <button type="submit">Transfer</button>
    </form>
  `);
});

// — Route: Process the form (POST) – CSRF-protected —
app.post('/transfer', doubleCsrfProtection, (req, res) => {
  res.send(`Transfer of ${req.body.amount} to ${req.body.recipient} successful.`);
});

app.listen(3000, () => {
  console.log('Server running on http://localhost:3000');
});
```

Start the server and open `http://localhost:3000/transfer` in a browser. Fill in the form and submit — it succeeds because the hidden `_csrf` field contains a valid token.

Now simulate a forged request without the token:

```
curl -X POST http://localhost:3000/transfer -d "recipient=attacker&amount=10000"
```

Output:

```
ForbiddenError: invalid csrf token
  at doubleCsrfProtection (...)
  ...
```

What is happening?

- `generateToken(req, res)` creates a token and sets a corresponding `__csrf` cookie on the response.
- The token is embedded in the form as a hidden input named `_csrf`.
- When the form is submitted, `doubleCsrfProtection` middleware reads the token from `req.body._csrf` (as configured in `getTokenFromRequest`) and compares it against the cookie value.
- A request from a malicious site would not have access to either the token value or the ability to set the matching cookie, so it fails.
- The `curl` request fails because it has neither the token nor the cookie.

8.3 CSRF with JSON APIs

For AJAX requests that send JSON instead of form data, the token is typically included as a custom header rather than a form field. The `getTokenFromRequest` configuration above already supports this:

```
req.headers['x-csrf-token'] .
```

A client-side JavaScript request would look like:

```
// Client-side JavaScript (browser)
const csrfToken = document.querySelector('meta[name="csrf-token"]').content;

fetch('/transfer', {
  method: 'POST',
  headers: {
    'Content-Type': 'application/json',
    'X-CSRF-Token': csrfToken
  },
  body: JSON.stringify({ recipient: 'alice', amount: 100 })
});
```

The server would serve the token in a `<meta>` tag on the initial page load:

```
// In the GET route that serves your HTML page
const csrfToken = generateToken(req, res);
res.send(`
  <html>
  <head>
    <meta name="csrf-token" content="${csrfToken}" />
  </head>
  <body>...</body>
</html>
`);
```

Note: In a typical production setup, the CSRF token is generated once when the page loads and reused for all AJAX requests during that session. The `csrf-csrf` package handles rotation and validation transparently.

9. Combining Validation and CSRF Protection

In a real application, both CSRF protection and input validation run on the same route. The CSRF check should come first — there is no point validating a request that was forged.

server.js (combined example)

```
const { body } = require('express-validator');
const validate = require('./middleware/validate');

app.post(
  '/transfer',
  doubleCsrfProtection, // 1. CSRF check first
  [ // 2. Then validate input
    body('recipient')
      .trim()
      .notEmpty().withMessage('Recipient is required.')
      .isLength({ min: 2 }).withMessage('Recipient name too short.'),
    body('amount')
      .isFloat({ min: 0.01 }).withMessage('Amount must be a positive number.')
      .toFloat()
  ],
  (req, res) => {
    // ...
  })
```

```

validate, // 3. Collect validation errors
(req, res) => { // 4. Business logic
  res.send(`Transfer of £${req.body.amount} to ${req.body.recipient} successful.`);
}
);

```

The middleware executes left to right: CSRF → validation chains → validation error check → handler. If any middleware calls `res.status(...).json(...)` or throws an error, the chain stops and the handler never runs.

Summary

Concept	Key Point
Server-side validation	Always required — client-side validation can be bypassed entirely
express-validator	Chainable validators and sanitisers that run as Express middleware; uses <code>body()</code> , <code>query()</code> , <code>param()</code>
Sanitisers	<code>.trim()</code> , <code>.escape()</code> , <code>.normalizeEmail()</code> , <code>.toInt()</code> — modify <code>req</code> data in place
Custom validators	<code>.custom((value, { req }) => { ... })</code> for application-specific rules; can be async
Reusable patterns	Separate rules into <code>validators/*.js</code> , use a generic <code>validate</code> middleware to check results
Joi	Schema-based alternative — defines the entire expected shape of an object; framework-agnostic
CSRF attacks	

	Exploit session cookies by tricking the browser into sending forged requests from a malicious site
CSRF tokens	Server-generated tokens embedded in forms or headers; verified on state-changing requests
csrf-csrf package	Double Submit Cookie pattern; replaces deprecated <code>csrf</code>
Middleware order	CSRF protection first, then validation, then business logic

Interview Questions

1. Why is server-side validation necessary even when client-side validation exists?

Expected answer: Client-side validation runs in the browser and can be bypassed by sending requests directly (e.g., via curl, Postman, or scripts). Server-side validation is the only layer you control entirely, so it is the only reliable enforcement point for data integrity and security.

2. What is the difference between a validator and a sanitiser in express-validator?

Expected answer: A validator checks a condition and records an error if it fails (e.g., `.isEmail()`). A sanitiser modifies the value in place (e.g., `.trim()` removes whitespace, `.toInt()` converts to a number). Sanitisers do not produce errors.

3. How does `.custom()` work in express-validator, and when would you use it?

Expected answer: `.custom()` accepts a function that receives the field value and a context object (containing `req`). It should return `true` for success or throw an error for failure. It can also return a Promise for async checks. Use it for rules that are not covered by built-in validators, such as password confirmation or database uniqueness checks.

4. Describe the reusable validation middleware pattern. Why is it beneficial?

Expected answer: Validation rules are extracted into dedicated files (e.g., `validators/userValidators.js`), and a generic `validate` middleware calls `validationResult(req)` to check for

errors. This separates concerns: rules are defined once, the error-checking logic is written once, and route handlers focus purely on business logic. It also produces a consistent error response format across all endpoints.

5. Compare express-validator and Joi. In what scenarios might you prefer one over the other?

Expected answer: `express-validator` integrates directly into Express middleware and can validate `body`, `query`, `params`, `headers`, and `cookies` independently. Joi defines a declarative schema for an entire object and is framework-agnostic. Prefer `express-validator` for tight Express integration; prefer Joi when schemas need to be reused outside Express or when a declarative style is preferred.

6. What is a CSRF attack, and why are session-based applications vulnerable?

Expected answer: A CSRF attack occurs when a malicious site tricks a user's browser into sending a request to a target application. Since the browser automatically includes cookies (including session cookies) with every request to the target domain, the server cannot tell the request is forged. The attacker exploits the authenticated session without knowing the user's credentials.

7. Explain the Double Submit Cookie pattern for CSRF protection.

Expected answer: The server sets a CSRF token in a cookie and also provides the same token to the client (in a hidden form field or response body). On form submission, the client sends both the cookie and the token in the request body or header. The server compares them — if they match, the request is legitimate. A malicious site cannot read or set cookies for a different domain, so it cannot forge a matching pair.

8. Should you apply CSRF protection to a REST API that uses JWT in the Authorization header? Why or why not?

Expected answer: Generally no. CSRF attacks exploit the browser's automatic cookie attachment. If authentication relies on a JWT in the `Authorization` header (which the browser does not send automatically), there is no cookie to exploit. However, if the JWT is stored in a cookie (rather than in memory or `localStorage`), CSRF protection is still necessary.

Practical Assignment: User Registration API with Full Validation

Objective: Build a `/api/users` endpoint that validates registration data using `express-validator`, returns structured error responses, and demonstrates sanitisation.

Setup

1. Create a new directory called `registration-api` and initialise a project:

```
mkdir registration-api && cd registration-api && npm init -y
```

2. Install dependencies:

```
npm install express express-validator
```

Requirements

1. Create the following folder structure:

```
registration-api/  
├── middleware/  
│   └── validate.js  
├── validators/  
│   └── userValidators.js  
├── routes/  
│   └── users.js  
├── app.js  
└── package.json
```

2. `middleware/validate.js` — Generic validation-runner middleware that returns errors in this format:

```
{  
  "status": "error",  
  "errors": [  
    { "field": "username", "message": "Username is required." }  
  ]  
}
```

3. `validators/userValidators.js` — Define `registerRules` that enforce:
 - `username` : required, trimmed, 3–20 characters, alphanumeric and underscores only
 - `email` : required, valid email, normalised
 - `password` : required, minimum 8 characters, at least one uppercase letter, at least one digit
 - `confirmPassword` : must match `password`
 - `age` : optional, if provided must be an integer between 13 and 120
 - `bio` : optional, if provided must be at most 500 characters, trimmed and escaped
4. `routes/users.js` — An Express Router with:
 - `POST /api/users` that applies rules, runs validation, and responds with the cleaned user data on success
5. `app.js` — Wire everything together and listen on port 3000.

Test Cases

Use `curl` or Postman to verify:

1. A valid registration returns `201` with sanitised user data (trimmed username, normalised email).
2. An empty body returns `400` with errors for all required fields.
3. A mismatched `confirmPassword` returns the custom error message.
4. A `bio` containing HTML tags (e.g., `<script>`) is escaped in the response.

Expected Output — Valid Request

```
curl -X POST http://localhost:3000/api/users -H "Content-Type: application/json" -d '{"username": " alice_dev ", "email": "Alice@Gmail.com", "password": "Secure1pass", "confirmPassword": "Secure1pass", "age": 28, "bio": " Full-stack developer "}'
```

Output:

```
{
  "status": "success",
  "user": {
    "username": "alice_dev",
    "email": "alice@gmail.com",
    "age": 28,
```

```
    "bio": "Full-stack developer"
  }
}
```

Expected Output — Invalid Request

```
curl -X POST http://localhost:3000/api/users -H "Content-Type: application/json" -d
'{"username": "a!", "email": "notanemail", "password": "weak", "confirmPassword": "different",
"age": 5}'
```

Output:

```
{
  "status": "error",
  "errors": [
    { "field": "username", "message": "Username must be 3-20 characters." },
    { "field": "username", "message": "Username may only contain letters, numbers, and underscores." },
    { "field": "email", "message": "Must be a valid email address." },
    { "field": "password", "message": "Password must be at least 8 characters." },
    { "field": "password", "message": "Password must contain at least one uppercase letter." },
    { "field": "password", "message": "Password must contain at least one number." },
    { "field": "confirmPassword", "message": "Password confirmation does not match." },
    { "field": "age", "message": "Age must be between 13 and 120." }
  ]
}
```

Bonus Challenge

Add a `POST /api/users/login` route with its own `loginRules` (email + password required). Add a custom async validator to the registration route that checks if the username is already taken by maintaining an in-memory array of registered users. If the username exists, the custom validator should reject with the message `"Username is already taken."`.